FAAMG at AraSeg Shared Task 2026: Cross-Architecture Ensembling for Punctuation-Free Arabic Sentence Segmentation

Gaser Elmasry[1] **Abdulrahman Elbedewy**[2] **Abdelrahman Atef**[3]
Fahd Seddik[4] **Mohamed Abdelmoniem**[2]
[1]American University in Cairo [2]University of Texas at Austin
[3]Independent Researcher [4]University of British Columbia
gaser.elmasry@aucegypt.edu abdulrahman.elbedewy@utexas.edu
ab.wael@outlook.com fahd.seddik@ubc.ca moabdelmoniem@utexas.edu

Abstract

Arabic sentence segmentation without punctuation or paragraph boundaries requires models to infer boundaries only from lexical, syntactic, and discourse context. We describe **FAAMG**'s submission in the closed track of **AraSeg 2026** Subtask 4 (NoPnx-NP), restricted to only 174 training documents for fine-tuning. Our system is a weighted ensemble of eight models based on AraBERT, SaT, and CAMeLBERT BiLSTM-CRF. The ensemble covers a variety of regularization, encoder capacity, and sequence modeling. The resulting system achieves 86.7 document-macro F1 on the Blind Test set ranking **fourth** on the leaderboard. Through leave-one-out ablations we show that the ensemble contributes complementary information over the individual models.

1 Introduction

Sentence segmentation is a foundational preprocessing step in many NLP pipelines where errors can affect downstream analysis. For Arabic, Elkholy et al. (2026a) show that more accurate sentence boundaries substantially improve dependency parsing. Many segmentation systems rely on punctuation, but it is an imperfect proxy for sentence boundaries. Punctuation may occur within sentences, occur at the end of sentences, be used inconsistently, or simply not be present (Read et al., 2012; Minixhofer et al., 2023; Frohmann et al., 2024).

Previous Arabic literature has examined punctuation-aware rules, Tunisian Arabic, unpunctuated text, and bilingual-document segmentation (Hadrich Belguith et al., 2005; Mekki et al., 2022; Alshanqiti et al., 2022; Al-Khdour et al., 2022). These studies address important parts of the problem, but not the combination of missing punctuation, missing paragraph structure, and broad genre variation considered by AraSeg (Elkholy et al., 2026b).

Subtask 4 (NoPnx-NP; No Punctuation, No Paragraphs) removes both punctuation and paragraph boundaries, requiring systems to infer whether a sentence boundary follows each token only using lexical, syntactic, and discourse context, which in turn creates ambiguity. For example, Arabic conjunctions and discourse markers may either continue the current sentence or introduce a new one. AraSeg also shows that boundary behavior does not transfer uniformly across its eight genres (Elkholy et al., 2026a). The closed track provides 174 training documents with 98,457 tokens, of which 10,247 are positive boundary positions, combining limited supervision with substantial class imbalance.

We explore comparisons of pretrained backbone, context length, training objective, regularization, and classification head. We identify AraBERTv02-base (Antoun et al., 2020) as the strongest tested backbone. Through an ensemble of the same model under different random seeds we improve the robustness of the system. Therefore, we combine multiple models that introduce complementary approaches such as SMART regularization (Jiang et al., 2020), AraBERT base and large, Segment any Text (SaT) (Frohmann et al., 2024), and a BiLSTM with Conditional Random Fields (CRF) (Lafferty et al., 2001) on CAMeLBERT (Inoue et al., 2021) representations. Our final system is an eight-model cross-architecture ensemble, where we fit the ensemble weights and a global decision threshold using development data. The ensemble allows weaker models to compensate for errors of stronger ones. Our contributions are:

- An ensemble of eight candidates that achieves 86.7 F1 on the Blind Test set, ranking fourth in the NoPnx-NP closed track.
- We show that seed averaging improves robustness, while cross-architecture ensembling provides complementary information that compensates for individual failure modes.

Proceedings of The Fourth Arabic Natural Language Processing Conference (ArabicNLP 2026) - Volume 2 Shared Tasks, pages 977–994
October 28-29, 2026 ©2026 Association for Computational Linguistics

2 Background

2.1 Shared Task Description

AraSeg evaluates Arabic sentence segmentation across eight genres and multiple punctuation/document-structure conditions (Elkholy et al., 2026a). We participate only in the closed track for Subtask 4 (NoPnx-NP) where punctuation and paragraph boundaries are removed (Elkholy et al., 2026b). Given a whitespace-tokenized document $x = (w_1, \ldots, w_n)$, the system predicts $y_i \in \{0, 1\}$ for each word, where $y_i = 1$ means that a sentence ends after w_i. No additional data other than the train set provided by the challenge is used for fine-tuning.

2.2 Related Work

Earlier sentence segmentation systems relied on rules, punctuation, and statistical boundary cues (Kiss and Strunk, 2006). However, recent multilingual systems reduce this dependence. Where's the Point? (WtP) learns punctuation-agnostic boundary representations through self-supervision (Minixhofer et al., 2023), while Segment Any Text (SaT) uses segmentation-specific pretraining and efficient adaptation (Frohmann et al., 2024). These target broad multilingual transfer rather than Arabic-specific systems.

Arabic-specific literature covers several segmentation settings. Similarly, earlier works use contextual rules over punctuation and a predefined list of Arabic particles, including coordinating conjunctions (Hadrich Belguith et al., 2005). Recent work studies sentence boundaries in Tunisian Arabic (Mekki et al., 2022), unpunctuated Arabic text (Alshanqiti et al., 2022), and bilingual documents (Al-Khdour et al., 2022). These studies address particular varieties, domains, or input conditions. By comparison, AraSeg evaluates sentence segmentation across eight Arabic genres and multiple punctuation and document-structure conditions (Elkholy et al., 2026a).

3 System Overview

We can describe our design for the system in the context of targeting three main challenges in NoPnx-NP: 1) **missing structural cues** motivate contextual token-level boundary prediction, 2) **limited and imbalanced supervision** motivates class-weighted training, regularization, and seed averaging, and 3) **genre variation** motivates candidates that differ in pretraining, encoder capacity, and document-level modeling. Table 1 summarizes the final candidate pool of models.

Candidate family	Count	Modeling role
AraBERT-base R-Drop	4	seed diversity
AraBERT-base SMART	1	regularization
AraBERT-large R-Drop	1	encoder capacity
SaT-12l + LoRA	1	segmentation pre-training
Cluster BiLSTM–CRF	1	register + sequence

Table 1: Candidate pool for ensemble fitting.

3.1 AraBERT Boundary Classification

For AraBERT-family candidates, documents are encoded with overlapping 256-subtoken windows and stride 128 (averaging logits in a moving window). Due to each AraSeg label indicating whether a boundary follows a whitespace token, we apply supervision to that token's final subword. In addition, a two-class linear head is trained with class-weighted cross-entropy, with class weights derived from the train label frequencies. We discuss further ablations in Section 5.2 and Appendices B–C about window size, subword alignment, loss function, and output head.

3.2 Heterogeneous Candidates

As shown in Table 1, we fine-tune AraBERTv02-base using R-Drop (Liang et al., 2021) ($\alpha = 0.5$) with seeds 13, 17, 23, and 31. A fifth AraBERTv02-base candidate uses SMART adversarial regularization with two perturbation steps. The AraBERTv02-large variant uses the same word-level boundary formulation with R-Drop, mainly to provide capacity diversity. A seventh candidate adapts the segmentation-pretrained SaT-12l with LoRA (Hu et al., 2022) (rank 64, $\alpha = 128$). The final candidate concatenates CAMeLBERT-MSA representations with a document embedding from K-Means ($k = 8$, $dim = 64$), then predicts boundaries with a one-layer BiLSTM and linear-chain CRF. K-Means is fitted only to training set document representations. We treat the resulting clusters as latent document-level features rather than expecting them to align one-to-one with the eight genres of AraSeg.

3.3 Weighted Probability Fusion

Each candidate produces a boundary probability $p_i^{(m)}$ for word i. We combine them as $p_i = \sum_{m=1}^{M} w_m p_i^{(m)}$, where $w_m \geq 0$ and $\sum_m w_m = 1$,

and predict $\hat{y}_i = \mathbf{1}[p_i \geq \tau]$. Each candidate receives a non-negative weight, which may be zero, and the weights sum to one. Afterwards, a single global decision threshold is applied (Appendix A).

4 Experimental Setup

4.1 Data, Split Usage, and Preprocessing

We use only the AraSeg NoPnx-NP closed track data. Table 2 reports the exact splits. All training used the train split. Development split is used for model selection, thresholds, and ensemble fitting. Open test set and Blind Test set are used for evaluation only, where labels for the latter are not released.

Split	Docs	Tokens	Boundaries	Role
Train	174	98,457	10,247	training
Dev	222	127,722	12,926	selection
Open Test	262	123,561	12,342	evaluation
Blind Test	212	96,798	–	official eval.

Table 2: NoPnx-NP data and how each split is used.

We keep the released AraSeg word tokens unchanged. We explored several preprocessing variants, including AraBERT-specific preprocessing, digit normalization, Latin-case handling, and number transformations, but did not observe a consistent gain in scores (detailed comparisons provided in Appendix B).

4.2 Training and Implementation

AraBERT R-Drop and AraBERT-large are trained for eight epochs with batch size 8, learning rate 2×10^{-5}, weight decay 0.01, warmup 0.1, gradient clipping 1, and bfloat16 precision. The SMART model uses AraBERTv02-base and trains for 12 epochs (details for the rest of the models in Appendix A). Experiments were run on H100 80GB GPUs. We use PyTorch (Paszke et al., 2019), Transformers (Wolf et al., 2020), PEFT (Mangrulkar et al., 2022), wtpsplit (Minixhofer et al., 2023; Frohmann et al., 2024), scikit-learn (Pedregosa et al., 2011), and the AraSeg evaluation code.

4.3 Evaluation and Selection

Precision, recall, and F1 are computed separately for every document and macro-averaged. Document-macro F1 is the primary metric for Subtask 4 (NoPnx-NP). Due to the training data being limited in the closed track, we utilize bootstrap probability to decide whether an approach improves over another with a threshold probability of $P_{\text{win}} \geq 97.5\%$.

5 Results

5.1 Official Performance

The team's best performing system (the eight-model ensemble) achieves **86.7** F1 with 85.8 precision and 88.6 recall on the Blind Test set, ranking **fourth** in the NoPnx-NP closed track. Its dev and open test F1 are 85.31 and 85.26, respectively. Other submissions by the team are described in Appendices H and I.

5.2 What Matters for the Base Model

We highlight the main results of our efforts in exploring alternative approaches and hyperparameters throughout this subsection. A 256-subtoken context improves over 128 by 0.50 F1 ($P_{\text{win}} = 98.8\%$). AraBERTv02-base is the strongest tested backbone as its mean F1 over three seeds is 83.86, compared to 83.40 for AraBERT-large and 80.65 for CAMeLBERT-MSA. We observe a slight improvement using SMART over R-Drop from 84.43 to 84.65 ($P_{\text{win}} = 97.8\%$).

During an audit of alternative loss functions, weighted cross-entropy and tuned focal loss (Lin et al., 2017) resulted in similar scores (84.09 vs. 84.10; $P_{\text{win}} = 53.2\%$). Finally, a linear output head outperforms CRF/BiLSTM-CRF alternatives (83.92 vs. 83.12–83.52). Continued train-only MLM adaptation, boundary augmentation, longer context, hard minimum-gap decoding, confidence weighting, and neural stacking resulted in either equal or worse performance.

Overall, several additions that seemed intuitive did not lead to consistent improvements (complete setup and results in Appendices B–E).

5.3 Why the Ensemble Helps

We explore the interactive mechanics of each model in the ensemble. Our study revealed that same-recipe seed averaging improves performance. In an experiment of aggregating the same approach across five seeds, individual models scored 84.05–84.42 dev F1 and 84.32–84.95 open test F1, while equal-logit averaging reached 84.74 and 85.56, respectively. This improvement was observed despite high correlation between the models' predictions in this experiment (Appendix D).

Further prediction correlation studies of SaT and the AraBERT family reveal that they are highly cor-

related ($r = 0.89$). However, an AraBERT+SaT mixture leads to improvement from 84.77/85.17 to 85.07/85.35 dev/open test F1 ($P_{\text{win}} = 99.7\%$). Diversity is not sufficient by itself as an exploration of AraELECTRA (Antoun et al., 2021) was competitive but turned out highly redundant ($r = 0.95$, $P_{\text{win}} = 70.5\%$), whereas QARiB (Abdelali et al., 2021) is less correlated ($r = 0.57$) but led to bad performance (51.9 F1; $P_{\text{win}} = 8.8\%$). We observe a similar pattern through a different genre-aware set of experiments (Appendix F). Therefore, useful members in an ensemble require both adequate individual quality and complementary predictions.

Ablation	Fixed Δ	Reweighted Δ	Necessary?
Full Ensemble (8 models)	**85.31 F1**	—	—
w/o SMART	−0.86	−0.08	✓
w/o AraBERT-large	−0.57	−0.17	✓
w/o SaT	−0.48	−0.22	✓
w/o R-Drop seeds (×3)	−0.20	≈ 0.00	✗
w/o BiLSTM	−0.20	≈ 0.00	✗

Table 3: Leave-one-out ablation (dev F1).

Final-component ablation. To test whether each ensemble member is actually necessary, we perform leave-one-out ablations. In Table 3, we measure this in two ways on the dev set: (1) removing the component while fixing the original ensemble weights, and (2) removing and re-searching the weights and threshold on the remaining models. SMART, AraBERT-large, and SaT each cause a drop in F1 even after reweighting, which means the ensemble cannot fully replace the information they contribute. On the other hand, removing the R-Drop models or the BiLSTM model causes a drop under fixed weights, but once the remaining models are reweighted, performance is fully recovered. Although this demonstrates that R-Drop and BiLSTM models are redundant relative to the rest of the ensemble on dev F1, the eight-model ensemble achieved better results on the Blind Test set (details in Appendix D.2).

5.4 Error Analysis

We analyze the submitted ensemble on all 222 dev documents (85.31 F1) and 262 open test documents (85.26 F1). Recall increases with sentence length, from 88.21% for sentences of at most nine tokens, 89.67% for 10–20 tokens, and 93.53% above 20.

Errors repeatedly occur around common function words and discourse transitions, including في (*fi*, 'in'), ثم (*thumma*, 'then'), من (*min*, 'from'), and وقد (*wa-qad*, 'and indeed'). Corpus statistics

Gold Sentence Length	Recall	Number of Boundaries
≤ 9 tokens	88.21%	7,772
10–20 tokens	89.67%	3,747
> 20 tokens	93.53%	1,407

Table 4: Boundary recall by gold sentence length.

show that these cues often occur at sentence boundaries, but direct lexical adjustments did not improve dev F1. Per-document F1 is neither correlated with document length ($r = 0.02$) nor boundary density ($r = 0.01$). Therefore, the remaining difficulty is better characterized by local boundary ambiguity rather than aggregate document properties. Appendix G reports full counts, token rankings, and boundary word frequencies.

6 Conclusion

FAAMG achieves 86.7 F1 on the Blind Test set and ranks fourth in the AraSeg NoPnx-NP closed track. The team's best submission combined a cross-architecture eight-model ensemble. Through our evaluation of different approaches, we observe improvements from averaging multiple seeds. Conversely, ablations on our final system show that SMART, AraBERT-large, and SaT contribute information that cannot be fully replaced by reweighting the remaining candidates. Our error analysis identifies short sentences and ambiguous local transitions as persistent challenges, whereas other approaches did not provide consistent gains.

Limitations. While our proposed approach demonstrates strong performance, we note two limitations driven by the constraints of the shared task. **First**, our study is strictly scoped to the AraSeg NoPnx-NP condition under the closed track, and given the restricted dataset scale, how well our conclusions transfer to punctuation-aware or paragraph-aware segmentation variants remains an area for future exploration. **Second**, to maximize predictive performance within the competition's strict time limits, our final system utilized a comprehensive 8-model ensemble; while effective for achieving top scores, our post-evaluation analysis suggests that this configuration might simply be scoring better on the Blind Test set, and time constraints led us to defer a deeper exploration of model pruning and efficiency trade-offs.

References

Ahmed Abdelali, Sabit Hassan, Hamdy Mubarak, Kareem Darwish, and Younes Samih. 2021. Pre-training BERT on Arabic tweets: Practical considerations. *arXiv preprint arXiv:2102.10684*.

Muhammad Abdul-Mageed, AbdelRahim Elmadany, and El Moatez Billah Nagoudi. 2021. ARBERT & MARBERT: Deep bidirectional transformers for Arabic. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 7088–7105, Online. Association for Computational Linguistics.

Nour Al-Khdour, Rebecca Jonsson, Ruba W Jaikat, Abdallah Nasir, Sara Alisis, Sara Qardan, and Eyas Shawahneh. 2022. Unlocking the value of bilingual translated documents with deep learning segmentation and alignment for Arabic. In *Proceedings of the 15th Biennial Conference of the Association for Machine Translation in the Americas (Volume 2: Users and Providers Track and Government Track)*, pages 341–359, Orlando, USA. Association for Machine Translation in the Americas.

Abdullah M. Alshanqiti, Sami Albouq, Ahmad B. Alkhodre, Abdallah Namoun, and Emad Nabil. 2022. Employing a multilingual transformer model for segmenting unpunctuated Arabic text. *Applied Sciences*, 12(20):10559.

Wissam Antoun, Fady Baly, and Hazem Hajj. 2020. AraBERT: Transformer-based model for Arabic language understanding. In *Proceedings of the 4th Workshop on Open-Source Arabic Corpora and Processing Tools, with a Shared Task on Offensive Language Detection*, pages 9–15, Marseille, France. European Language Resource Association.

Wissam Antoun, Fady Baly, and Hazem Hajj. 2021. AraELECTRA: Pre-training text discriminators for Arabic language understanding. In *Proceedings of the Sixth Arabic Natural Language Processing Workshop*, pages 191–195, Kyiv, Ukraine (Virtual). Association for Computational Linguistics.

Alexis Conneau, Kartikay Khandelwal, Naman Goyal, Vishrav Chaudhary, Guillaume Wenzek, Francisco Guzmán, Edouard Grave, Myle Ott, Luke Zettlemoyer, and Veselin Stoyanov. 2020. Unsupervised cross-lingual representation learning at scale. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 8440–8451, Online. Association for Computational Linguistics.

Mohammed Elkholy, Khalid N. Elmadani, Nizar Habash, and Bashar Alhafni. 2026a. Arabic sentence segmentation across genres and punctuation conditions. *Preprint*, arXiv:2606.08025.

Mohammed Elkholy, Khalid N. Elmadani, Nizar Habash, and Bashar Alhafni. 2026b. Arabic sentence segmentation shared task 2026. In *Proceedings of the Fourth Arabic Natural Language Processing Conference*, Budapest, Hungary. Association for Computational Linguistics.

Markus Frohmann, Igor Sterner, Ivan Vulić, Benjamin Minixhofer, and Markus Schedl. 2024. Segment any text: A universal approach for robust, efficient and adaptable sentence segmentation. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 11908–11941, Miami, FL, USA. Association for Computational Linguistics.

Lamia Hadrich Belguith, Leila Baccour, and Mourad Ghassan. 2005. Segmentation de textes arabes basée sur l'analyse contextuelle des signes de ponctuations et de certaines particules. In *Actes de la 12ème conférence sur le Traitement Automatique des Langues Naturelles. Articles courts*, pages 451–456, Dourdan, France. ATALA.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-rank adaptation of large language models. In *The Tenth International Conference on Learning Representations (ICLR 2022)*, Virtual Event. OpenReview.net.

Go Inoue, Bashar Alhafni, Nurpeiis Baimukan, Houda Bouamor, and Nizar Habash. 2021. The interplay of variant, size, and task type in Arabic pre-trained language models. In *Proceedings of the Sixth Arabic Natural Language Processing Workshop*, pages 92–104, Kyiv, Ukraine (Virtual). Association for Computational Linguistics.

Haoming Jiang, Pengcheng He, Weizhu Chen, Xiaodong Liu, Jianfeng Gao, and Tuo Zhao. 2020. SMART: Robust and efficient fine-tuning for pre-trained natural language models through principled regularized optimization. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 2177–2190, Online. Association for Computational Linguistics.

Tibor Kiss and Jan Strunk. 2006. Unsupervised multilingual sentence boundary detection. *Computational Linguistics*, 32(4):485–525.

John D. Lafferty, Andrew McCallum, and Fernando C. N. Pereira. 2001. Conditional random fields: Probabilistic models for segmenting and labeling sequence data. In *Proceedings of the Eighteenth International Conference on Machine Learning (ICML 2001)*, pages 282–289, Williamstown, MA, USA. Morgan Kaufmann.

Xiaoya Li, Xiaofei Sun, Yuxian Meng, Junjun Liang, Fei Wu, and Jiwei Li. 2020. Dice loss for data-imbalanced NLP tasks. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 465–476, Online. Association for Computational Linguistics.

Xiaobo Liang, Lijun Wu, Juntao Li, Yue Wang, Qi Meng, Tao Qin, Wei Chen, Min Zhang, and Tie-Yan Liu. 2021. R-drop: Regularized dropout for neural networks. In *Advances in Neural Information Processing Systems 34 (NeurIPS 2021)*, pages 10890–10905.

Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. 2017. Focal loss for dense object detection. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 2999–3007.

Sourab Mangrulkar, Sylvain Gugger, Lysandre Debut, Younes Belkada, Sayak Paul, Benjamin Bossan, and Marian Tietz. 2022. PEFT: State-of-the-art parameter-efficient fine-tuning methods. `https://github.com/huggingface/peft`.

Asma Mekki, Inès Zribi, Mariem Ellouze, and Lamia Hadrich Belguith. 2022. Sentence boundary detection of various forms of Tunisian Arabic. *Language Resources and Evaluation*, 56(1):357–385.

Benjamin Minixhofer, Jonas Pfeiffer, and Ivan Vulić. 2023. Where's the point? self-supervised multilingual punctuation-agnostic sentence segmentation. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 7215–7235, Toronto, Canada. Association for Computational Linguistics.

Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, and 2 others. 2019. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, pages 8024–8035. Curran Associates, Inc.

Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. 2011. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12:2825–2830.

Jonathon Read, Rebecca Dridan, Stephan Oepen, and Lars Jørgen Solberg. 2012. Sentence boundary detection: A long solved problem? In *Proceedings of COLING 2012: Posters*, pages 985–994, Mumbai, India. The COLING 2012 Organizing Committee.

Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, and 3 others. 2020. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, Online. Association for Computational Linguistics.

Appendix Contents

Part I: Primary System Supplementary Material

A Final System Configuration 6
B Input, Preprocessing, and Context 8
C Additional Model Design Results 9
D Ensemble Evidence 10
E Decoding and Post-processing Controls . . . 12
F Register Specialization 12
G Final-System Error Analysis 13

Part II: Additional FAAMG Submissions

H Second FAAMG Closed-Track Submission 14
I Third FAAMG Closed-Track Submission . . 17

Part I: Primary System Supplementary Material

A Final System Configuration

This section reports the complete configuration of each of the eight ensemble candidates summarized in Table 1, in the same order, together with the procedure used to fit the ensemble weights and decision threshold.

A.1 AraBERTv02-base with R-Drop

Four candidates share one configuration and differ only in random seed (13, 17, 23, and 31). Each document is encoded with overlapping 256-subtoken windows and stride 128. Supervision and prediction are attached to the final subword of every AraSeg word. A two-class linear head is trained with weighted cross-entropy and an R-Drop consistency term ($\alpha = 0.5$) (Liang et al., 2021). This windowing, subword alignment, and linear head carry over unchanged to the SMART and AraBERT-large candidates described next. Table 5 lists the complete configuration.

A.2 AraBERTv02-base with SMART

The SMART candidate is a full fine-tune of `aubmindlab/bert-base-arabertv02` (seed 13) that keeps this configuration in full and changes only the regularizer. It replaces the R-Drop consistency term with SMART's adversarial smoothness penalty (Jiang et al., 2020), a symmetric-KL

Setting	Value
Model	AraBERTv02-base
Seeds	13, 17, 23, 31
Window / stride	256 / 128 subtokens
Word representation	final subword
Objective	weighted cross-entropy + R-Drop ($\alpha = 0.5$)
Epochs	8
Batch size / accumulation	8 / 1 (effective 8)
Learning rate	2×10^{-5}
Weight decay / warmup	0.01 / 0.10
Gradient clipping	1.0
Precision	bfloat16
Optimizer / schedule	AdamW, linear warmup and decay

Table 5: Configuration of the four AraBERTv02-base R-Drop candidates.

term computed between clean and adversarially perturbed predictions. It also trains for more epochs than the R-Drop family (12 rather than 8). Table 6 lists the complete configuration.

Setting	Value
Model / seed	AraBERTv02-base / 13
Window / stride	256 / 128 subtokens
Word representation	final subword
Task objective	weighted cross-entropy
SMART weight	$\lambda = 2.0$
Adversarial steps	2
Adversarial LR	0.03
Perturbation radius	0.10
Initialization magnitude	0.10
Epochs	12
Batch size / accumulation	8 / 1 (effective 8)
Learning rate	2×10^{-5}
Weight decay / warmup	0.01 / 0.10
Gradient clipping	1.0
Optimizer / schedule	AdamW, linear warmup and decay

Table 6: Configuration of the SMART candidate.

A.3 AraBERTv02-large

The capacity-diverse candidate is a full fine-tune of AraBERTv02-large (seed 13) that again reuses the R-Drop configuration exactly, this time on a larger backbone. Its role in the ensemble is to add encoder capacity rather than a different regularizer or objective. Table 7 lists the complete configuration.

A.4 SaT-12l with LoRA

The remaining two candidates depart from this AraBERT configuration. The first uses `segment-any-text/sat-12l` with the `facebookAI/xlm-roberta-base` tokenizer (Conneau et al., 2020), a backbone already pretrained

Setting	Value
Model / seed	AraBERTv02-large / 13
Window / stride	256 / 128 subtokens
Word representation	final subword
Objective	weighted cross-entropy + R-Drop ($\alpha = 0.5$)
Epochs	8
Batch size / accumulation	8 / 1 (effective 8)
Learning rate	2×10^{-5}
Weight decay / warmup	0.01 / 0.10
Gradient clipping	1.0
Precision	bfloat16
Optimizer / schedule	AdamW, linear warmup and decay

Table 7: Configuration of the AraBERTv02-large candidate.

Setting	Value
Model / seed	SaT-12l / 42
Tokenizer	XLM-R base
Window / stride	256 / 128 subtokens
LoRA rank / α	64 / 128
LoRA dropout	0.10
LoRA targets	query, key, value, dense
Objective	class-balanced cross-entropy
Class weights	approximately 0.558 / 4.805 (negative / positive)
Epochs	10
Batch size / accumulation	4 / 4 (effective 16)
Learning rate	1×10^{-4}
Weight decay / warmup	0.01 / 0.10
Gradient clipping	1.0
Precision	bfloat16
Optimizer / schedule	AdamW, linear warmup and decay

Table 8: Configuration of the submitted SaT-12l+LoRA candidate.

specifically for segmentation, unlike the general-purpose encoder used by the AraBERT candidates. Because SaT is already pretrained for this task, we adapt it with LoRA (Hu et al., 2022) rather than updating its full parameter set. Low-rank adapters are inserted into the query, key, value, and dense attention projections. Table 8 reports the complete configuration.

A.5 Cluster-conditioned BiLSTM–CRF

The final candidate is the most structurally distinct, combining a continued CAMeLBERT-MSA encoder with a document-level cluster embedding and a sequence layer. Document-level representations from the train split are clustered with k-means ($k = 8$, seed 42, 10 initializations). The resulting cluster ID is represented as a 64-dimensional embedding. For each word, this embedding is concatenated with the CAMeLBERT-MSA representa-

tion and passed through a one-layer bidirectional LSTM, a linear emission layer, and a linear-chain CRF.

The CAMeLBERT-MSA encoder is first continued with masked-language modeling on train documents only. This continuation stage uses 10 epochs at batch size 8 with accumulation 4, learning rate 2×10^{-5}, weight decay 0.01, and warmup 0.10, with masking probability 0.20, maximum length 512, and seed 13. The clustering and all subsequent fitting use only train data. Table 9 lists the complete configuration.

Setting	Value
Encoder	CAMeLBERT-MSA
Head-training seed	42
Window / stride	256 / 128 subtokens
Number of clusters	8
Cluster embedding	64 dimensions
BiLSTM	1 layer, hidden size 256
Output layer	linear emissions + CRF
Objective	CRF negative log-likelihood
Positive-label scale	8.608 (Train negative/positive ratio)
Epochs	10
Batch size / accumulation	4 / 4 (effective 16)
Encoder learning rate	2×10^{-5}
Head learning rate	2×10^{-4}
Weight decay / warmup	0.01 / 0.10
Gradient clipping	1.0
Precision	bfloat16

Table 9: Configuration of the cluster-conditioned BiLSTM–CRF candidate.

A.6 Probability Ensemble

Combining the eight candidates' boundary probabilities requires fitting a non-negative weight per candidate and a single decision threshold. The seed-23 R-Drop candidate receives a near-zero weight in the submitted solution, below 0.0001 (Table 10). Its individual leave-one-out effect is negligible. Removing it alone leaves fixed-weight dev F1 unchanged and changes reoptimized dev F1 by only 0.02 points, well within the ensemble's own search-noise floor (Table 20, Section D.2). The final threshold is 0.55, with no minimum-gap rule applied.

The search samples 20,000 weight vectors from a $\mathrm{Dirichlet}(1, \ldots, 1)$ distribution and evaluates thresholds from 0.10 to 0.90 in steps of 0.05. The best vector is then refined with 4,000 Gaussian perturbations (standard deviation 0.05 per coordinate). Negative coordinates are clipped to zero and weights renormalized before searching thresholds from 0.05 to 0.95 in steps of 0.01. Dev document-macro F1 is the objective throughout the search.

Candidate	Weight
AraBERT R-Drop, seed 13	0.0254
AraBERT R-Drop, seed 17	0.0645
AraBERT R-Drop, seed 23	< 0.0001
AraBERT R-Drop, seed 31	0.0026
AraBERT SMART	0.3209
AraBERT-large	0.2910
SaT-12l + LoRA	0.1664
Cluster BiLSTM–CRF	0.1293

Table 10: Submitted ensemble weights.

All candidates were trained on NVIDIA H100 80GB GPUs, using PyTorch and Hugging Face Transformers, with PEFT for the SaT adaptation and scikit-learn for document clustering.

B Input, Preprocessing, and Context

B.1 Preprocessing and Normalization

The final system keeps the released AraSeg token sequence unchanged. We tested both model-specific preprocessing and targeted normalization before deciding not to add a normalization stage.

Standard AraBERT preprocessing. An early AraBERTv02-base comparison used a two-class linear head with final-subword alignment and the same linear warmup and decay configuration in both conditions. Leaving the AraSeg words unchanged reached 82.62 dev F1 and 83.35 open test F1, while applying the standard AraBERT preprocessing utility reached 82.20 and 83.14, respectively. The comparison therefore provides no evidence that AraBERT's preprocessing improves this task.

Input treatment	Dev F1	Open F1
Unmodified	**82.62**	**83.35**
Standard AraBERT preprocessing	82.20	83.14

Table 11: Matched AraBERTv02-base preprocessing comparison.

Targeted transformations. A second test applies transformations to the same five-checkpoint ensemble while keeping the threshold fixed at 0.25. Digit normalization changes 146 dev tokens, Latin lowercasing 132, and number-length mapping 687. None improves dev F1 (Table 12).

Input	Changed	Dev F1	Open F1	Δ
Unmodified	–	**83.729**	84.382	–
Digit normalization	146	83.727	84.383	-0.002
Latin lowercasing	132	83.723	**84.406**	-0.006
Number-length mapping	687	83.669	84.316	-0.060

Table 12: Inference-only preprocessing comparison. Δ is in dev F1 relative to the unmodified input.

A separate seed-13 retraining comparison gives the same overall conclusion, though it illustrates why we avoid claiming that unmodified input always wins numerically. Digit normalization reaches 83.01 dev F1 and 83.55 open test F1, versus 82.98 and 83.74 for the unmodified run. Latin lowercasing reaches 82.62/83.31 and number-length mapping 82.61/83.17 dev/open test F1. The small and inconsistent changes do not support a general normalization rule.

A corpus-level inspection also cautions against aggressive spelling repair. Only 475 token occurrences, about 0.136% of the labeled corpus, fall into mechanically repairable spelling pairs. Several are systematic source forms rather than random errors. For example, ءامنوا *'āmanū* occurs 81 times compared with آمنوا *'āmanū* three times, and الءاخرة *al-'ākhira* occurs 25 times compared with الآخرة *al-'ākhira* nine times. Across these pairs, the less-standard forms use more AraBERT WordPieces on average (3.07 vs. 1.63), but automatic replacement could remove source or register information. We therefore preserve the observed forms.

B.2 Subword Alignment

AraSeg labels are defined after whitespace-delimited words, while the encoders operate on subwords. We compared three ways to connect the two levels using the same CAMeLBERT-mix boundary classifier with cross-entropy, changing only the word-to-subword alignment. Using the final subword performs best (Table 13). This motivates the final-subword convention used by the AraBERT-family models.

Word-to-subword alignment	Dev F1
Final subword	**77.34**
Label every subword	77.08
First subword	76.94

Table 13: Matched CAMeLBERT-mix comparison of subword-label alignment.

We additionally tested mean/max pooling, first–mean–last fusion, attention pooling, and contextual pooling under different architectures. Because these additional experiments vary in more than the alignment rule, their absolute scores are not combined with Table 13 as a single ablation.

B.3 Context Length and Coverage

The strongest controlled context study uses the same AraBERTv02-base classifier and changes only maximum context length. The first sweep uses the baseline configuration, and the second repeats the context comparison after optimization. In both cases, 256 subtokens is best (Table 14). The 256-vs.-128 dev gain is 0.50 F1 points in the baseline sweep ($P_{\text{win}} = 98.8\%$) and 0.66 after optimization ($P_{\text{win}} = 98.8\%$).

Training recipe	128	256	384	512
Baseline	83.42	**83.92**	82.91	82.77
Optimized	83.44	**84.10**	83.23	83.29

Table 14: Controlled dev F1 comparison across maximum subtoken lengths.

The result also appears under the R-Drop and weighted cross-entropy configuration used by the final AraBERT family. A 256/128 window and stride reaches 84.18/84.25 dev/open test F1, compared with 83.58/83.79 for 384/192 and 83.29/83.79 for 512/256. The corpus itself makes 256 a reasonable capacity. 81.82% of sentences contain at most 16 words, 97.33% at most 32, and 99.997% of sentences fit within 256 subtokens for the inspected tokenizers. These coverage statistics explain why very long windows are rarely required, but the controlled F1 comparisons, not coverage, determine the selected context.

C Additional Model Design Results

C.1 Backbone Comparison

We compare seven Arabic pretrained encoders under the same three-seed boundary-classification protocol, including ARBERT and MARBERT (Abdul-Mageed et al., 2021). Seeds 13, 17, and 23 are used for every backbone, and the model head, data, and evaluation procedure are held fixed. Table 15 reports the individual dev scores and their mean.

AraBERTv02-base exceeds AraBERTv02-large by 0.45 F1 points ($P_{\text{win}} = 98.9\%$). This does

Backbone	s13	s17	s23	Mean [95% CI]
AraBERTv02-base	83.92	83.77	83.88	**83.86 [83.21,84.50]**
AraBERTv02-large	83.76	83.08	83.37	83.40 [82.69,84.11]
CAMeLBERT-MSA	80.77	80.56	80.61	80.65 [79.89,81.42]
CAMeLBERT-mix	79.08	79.09	79.37	79.18 [78.37,80.01]
CAMeLBERT-CA	77.41	77.02	77.34	77.26 [76.37,78.14]
ARBERT	72.11	72.13	71.77	72.00 [71.20,72.83]
MARBERT	69.20	69.15	69.21	69.19 [68.29,70.11]

Table 15: Controlled three-seed backbone comparison on dev.

Variant	Mean Dev F1	Pairwise P_{win}
Focal loss	**84.10**	0.5318
Weighted cross-entropy	84.09	–
SMART	**84.65**	0.9782
R-Drop	84.43	–
No boundary augmentation	**84.65**	0.9745
Boundary augmentation	84.40	–

Table 16: Controlled comparisons. P_{win} compares focal with weighted CE, SMART with R-Drop, and no augmentation with augmentation.

not make the large model unnecessary. Section D shows that it contributes complementary probabilities when combined with other models.

C.2 Losses and Regularization

The regularization study uses the controlled AraBERTv02-base configuration and confirms close comparisons over three seeds. Focal loss and weighted cross-entropy are effectively tied, whereas SMART improves over R-Drop (Table 16). Boundary augmentation is also tested with the SMART configuration and is weaker than leaving augmentation off.

The controlled regularization comparison uses independently tuned settings. R-Drop uses $\alpha = 1.7127$, while SMART uses $\lambda = 4.7768$, adversarial learning rate 0.0812, $\epsilon = 0.0282$, initialization magnitude 0.0196, and three perturbation steps. These settings are specific to the controlled comparison. Appendix A reports the configurations used by the submitted ensemble.

Through an exploration of alternative loss functions, we reach 81.79 dev F1 using Dice loss (Li et al., 2020). Continued masked-language-model adaptation on train documents also shows no meaningful gain. Off-the-shelf AraBERT averages 83.86 F1 across three seeds, versus 83.81 after train-only adaptation.

SaT adaptation. SaT itself proposes LoRA-based domain adaptation (Frohmann et al., 2024).

Head	Dev F1	Δ	P_{win}
Linear	**83.92**	–	–
CRF	83.52	-0.41	–
BiLSTM–CRF, h=128	83.38	-0.55	0.0037
BiLSTM–CRF, h=256	83.37	-0.55	0.0048
BiLSTM–CRF, h=384	83.12	-0.80	0.0001
BiLSTM–CRF, h=512	83.52	-0.40	0.0382

Table 17: Dev F1 for output-head alternatives. P_{win} is the probability that the structured alternative beats the linear head.

We additionally compare parameter-efficient adaptation with full fine-tuning on NoPnx-NP. A 30-trial SaT-12l+LoRA search spans 76.95–79.57 dev F1. The selected LoRA setting reaches 79.57, with a three-seed mean of approximately 79.54 dev and 79.00 open test F1. Full fine-tuning at learning rate 5×10^{-5} reaches 79.57 at seed 13. The direct comparison therefore finds no meaningful advantage from updating the full SaT backbone for this task, supporting LoRA as the more parameter-efficient adaptation strategy. This adaptation comparison uses its own tuned LoRA configuration. The submitted SaT configuration is given in Appendix A.

C.3 Output Heads

The head comparison uses AraBERTv02-base with the controlled 256-subtoken context configuration. The encoder, data, and training setup are held fixed while the prediction head changes. A simple linear classifier outperforms a linear-chain CRF and all tested BiLSTM–CRF hidden sizes (Table 17).

The structured models do change local boundary patterns. The proportion of predicted adjacent short-boundary pairs is 0.2157 for the linear head, 0.2140 for the CRF, and 0.1968 for the BiLSTM–CRF, compared with 0.1984 in the gold labels. The BiLSTM–CRF moves this structural statistic toward the target distribution, but the lost true boundaries outweigh the removed spurious ones, and the improvement in local structure therefore does not translate into higher document-macro F1.

D Ensemble Evidence

D.1 Seed Averaging

To isolate random-seed effects, we use a separate same-architecture AraBERTv02-base experiment in which only the seed changes. This study uses 510-subtoken windows with stride 128, focal training with whole-word masking, a fixed training

Seed-only system	Dev F1	Open F1
Seed 13	84.41	84.95
Seed 42	84.32	84.90
Seed 43	84.42	84.44
Seed 44	84.05	84.32
Seed 45	84.09	84.69
Equal-logit five-seed ensemble	**84.74**	**85.56**

Table 18: Matched seed-only experiment. The ensemble improves over every individual run on dev.

budget, and seeds 13, 42, 43, 44, and 45. All other model and optimization choices are shared. Individual dev/open test F1 ranges from 84.05–84.42/84.32–84.95, while equal-logit averaging reaches 84.74/85.56 (Table 18). The gain therefore cannot be attributed to a changed architecture or training configuration.

A second experiment tests whether ensemble diversity is better sourced from random seeds or from ordinary fold bagging when only 174 training documents are available. It uses a CAMeLBERT-MSA replication trained for a fixed budget of 380 updates. Five models trained on the complete train split are compared with five fold models, each trained on roughly 80% of the documents. The full-data seed ensemble reaches 80.50 dev and 81.92 open test F1, compared with 79.98 and 81.64 for fold bagging.

D.2 Final-System Component Ablation

We ablate the submitted ensemble in two ways. In the **fixed-weight removal**, the selected component is removed, the remaining submitted weights are renormalized, and the submitted threshold of 0.55 is kept fixed. This measures how much the submitted mixture depends on that component. In the **reoptimized removal**, the component is removed and the same dev-only weight and threshold search used for the full system is rerun. This tests whether the remaining candidates can compensate for the missing model.

For fixed-weight removals we use 10,000 paired bootstrap resamples of the 222 dev documents. Table 20 reports the resulting fixed-weight and reoptimized F1 change for each individually removed member, and Table 19 reports bootstrap confidence intervals and win probabilities for the fixed-weight removal of each component family. Negative Δ means that removal lowers F1.

The fixed-weight results show that SMART, AraBERT-large, and SaT are the largest load-bearing components of the submitted mixture, and the bootstrap intervals confirm that each of these three losses is reliably different from zero. Within the R-Drop seed family, most of the fixed-weight contribution comes from a single replica: seed 17 alone accounts for a 0.20-point loss when removed, while seeds 13, 23, and 31 each change F1 by 0.04 points or less. After reoptimization, the remaining AraBERT-family models can largely compensate for the additional seed replicas, whose individual reoptimized effects are small and mixed in sign, and the other candidates can compensate for the cluster model. SMART, AraBERT-large, and SaT still leave losses of 0.08, 0.17, and 0.22 F1 points after reoptimization, respectively. The weight-search variation measured for an unchanged member set is about 0.03 F1 points. These residual losses are therefore larger than the search noise observed in this procedure.

Removed component	95% CI for Δ	P_{win}
Additional R-Drop seeds	[-0.42,-0.01]	0.980
SMART	[-1.32,-0.45]	0.9999
AraBERT-large	[-0.87,-0.27]	0.9999
SaT + LoRA	[-0.82,-0.19]	0.9998
Cluster BiLSTM–CRF	[-0.40,-0.03]	0.991

Table 19: Paired-document bootstrap evidence for the fixed-weight ablations (10,000 resamples). P_{win} is the probability that the full system outperforms the ablated system.

Removed member	Fixed Δ	Reoptimized Δ
R-Drop seed 13	-0.04	+0.02
SMART	-0.86	-0.08
R-Drop seed 17	-0.20	-0.03
R-Drop seed 23	0.00	+0.02
R-Drop seed 31	-0.01	-0.04
AraBERT-large	-0.57	-0.17
SaT-12l + LoRA	-0.48	-0.22
Cluster BiLSTM–CRF	-0.20	0.00

Table 20: Individual-member leave-one-out changes in dev F1 points.

D.3 Quality and Diversity

AraBERT candidates trained under the same configuration have pairwise dev probability correlations of approximately 0.975–0.985. SaT is less redundant, with mean correlation 0.89 against six AraBERT-family candidates. In a controlled ensemble comparison, adding AraBERT-large to a five-seed AraBERT+SaT mixture raises dev/open test F1 from 84.77/85.17 to 85.07/85.35

($P_{\text{win}} = 99.7\%$). Adding SaT to a six-member AraBERT/AraBERT-large mixture raises dev/open test F1 from 85.10/85.06 to 85.31/85.20.

Two rejected candidates clarify the role of diversity. AraELECTRA is reasonably strong but highly correlated with the AraBERT family ($r = 0.95$) and gives no reliable gain ($P_{\text{win}} = 70.5\%$). QARiB is much less correlated ($r = 0.57$) but has only 51.9 dev F1 and also fails the inclusion test ($P_{\text{win}} = 8.8\%$). A candidate therefore needs both usable standalone quality and errors that are not already represented by the ensemble.

E Decoding and Post-processing Controls

The submitted system uses a single global threshold after probability fusion. We tested several alternatives before keeping this simple decoder. Because some controls were run with different strong probability models, each comparison below is interpreted against its own stated baseline rather than as one joint ablation.

Minimum gap. For the 256/128 AraBERT R-Drop classifier, minimum allowed boundary gaps of 0, 2, 3, and 4 words were compared on dev. A gap of 0 is selected. The submitted system therefore does not enforce a hard minimum sentence length.

Confidence weighting. Adaptive weighting based on prediction confidence reaches 85.20 dev and 84.97 open test F1, compared with 85.26 and 85.07 for the corresponding static probability fusion. Confidence therefore does not improve this strong ensemble.

Neural stacking. A neural stacker trained with grouped five-fold document-level cross-validation reaches 84.50 dev F1, below the corresponding static ensemble at 85.10. The additional meta-model does not improve the held-out document predictions.

Lexical marker adjustment. We also tested train-derived lexical offsets that slightly raise or lower a boundary logit according to how often selected Arabic tokens occur near boundaries in train. On a strong R-Drop AraBERT model, the unadjusted dev F1 is 84.18 and the adjusted value is 84.15. On the SMART model, the corresponding values are 84.63 and 84.62. Direct lexical correction therefore does not improve the dev metric even though the same marker words appear frequently in the error analysis.

Precision–recall effect of structural decoding. An earlier five-model probability ensemble illustrates why structured cleanup can look plausible without increasing F1. A gap-aware decoder reduces false positives from 2,697 to 2,547, but false negatives increase from 1,663 to 1,755. In other words, it removes 150 false boundaries while losing 92 additional true boundaries. This is the precision–recall trade-off behind the small gains or losses seen with structural post-processing. The stronger submitted probability model selects simple thresholding instead.

F Register Specialization

We test whether document-level specialization helps when the corpus contains heterogeneous registers, that is, distinct varieties of Arabic tied to genre and communicative purpose, such as Classical Quranic text, educational material, and expository news prose. CAMeLBERT-MSA document embeddings are computed for train documents and clustered with k-means ($k = 8$, seed 42, 10 initializations). The same fitted clustering is then used to assign dev and evaluation documents. One cluster is a coherent Quranic group containing 18 dev and 31 open test documents.

For this cluster, we compare the routing-study baseline with a CAMeLBERT-CA expert, whose pretraining is better matched to Classical Arabic. A route is then tuned on dev. Documents assigned to the Quranic cluster blend the routing-study baseline with CAMeLBERT-CA at an expert weight of 0.70 and a decision threshold of 0.41, while all other documents use the baseline unchanged.

System	Dev ($n = 18$)	Open ($n = 31$)
Routing baseline	90.20	81.12
CAMeLBERT-CA	93.73	88.17
Baseline + route	**93.89**	**88.36**

Table 21: F1 on the Quranic cluster. "Baseline + route" blends the routing-study baseline with the CAMeLBERT-CA expert for this cluster.

System	Whole Dev	Whole Open
Routing baseline	85.30	85.29
Baseline + Quranic route	**85.60**	**86.15**

Table 22: Effect of routing the Quranic cluster.

Cluster	Train / Dev docs	Baseline	Specialist
2	29 / 44	**84.46**	82.42
3	44 / 76	**82.61**	81.08
7	59 / 71	**87.77**	86.89

Table 23: Dev document-macro F1 for three non-Quranic cluster specialists.

The result is specific to a coherent register rather than to clustering itself. We repeated the same idea for three other clusters using continued same-backbone specialists. Cluster 2 mainly contains educational and quiz documents, Cluster 3 is a heterogeneous mixture, and Cluster 7 is dominated by expository and news text. All three specialists are weaker than the corresponding routing baseline on dev and receive zero routing weight (Table 23).

Cluster	Open docs	Baseline	Specialist
2	40	**89.89**	87.99
3	82	**81.45**	80.61
7	82	**88.22**	87.51

Table 24: Open Test document-macro F1 for the same specialists. Open Test is used only for evaluation.

The supported conclusion is therefore narrower than "genre experts help." A genuinely register-matched pretrained representation can be valuable for a coherent domain, while simply fine-tuning separate experts for arbitrary clusters does not reliably help. A global ninth-member blend of the routing-study baseline also assigns CAMeLBERT-CA zero weight.

G Final-System Error Analysis

The submitted weighted ensemble is evaluated with its fixed weights and a threshold of 0.55 on all 222 dev documents. It obtains 85.31 Dev and 85.26 Open Test document-macro F1. Aggregate boundary counts are 11,532 true positives, 2,521 false positives, and 1,394 false negatives. These counts correspond to corpus-level boundary precision 82.06%, recall 89.22%, and F1 85.49%. These corpus metrics are descriptive and are distinct from the official document-macro score.

G.1 Sentence Length and Document Properties

Table 4 groups each *true* boundary by the length of the gold sentence that ends there. Because the grouping is defined only for true boundaries, recall is the natural conditional metric. A false positive occurs inside a gold sentence and does not have an analogous "gold sentence ending at the predicted boundary." False positives are therefore analyzed separately below rather than assigned an artificial sentence-length precision bin.

Recall rises smoothly with sentence length, showing that short-sentence difficulty is not merely a consequence of short sentences being numerous. At the document level, token count has Pearson correlation $r = 0.02$ with document F1 and boundary density has $r = 0.01$. Aggregate document size and boundary rate therefore explain little of the remaining variation.

G.2 Frequent Lexical Contexts

For false negatives, we examine which token begins the sentence after the missed boundary. For false positives, we examine which token immediately precedes a boundary predicted where no gold boundary exists. Tables 25 and 26 show the ten most frequent contexts for each error type.

Rank	Missed sentence start	English gloss
1	في *fī*	in
2	ثم *thumma*	then
3	من *min*	from
4	بن *bin*	son of
5	لا *lā*	no / not
6	ولا *wa-lā*	nor / and not
7	دون *dūna*	without
8	وقد *wa-qad*	and indeed
9	وما *wa-mā*	and what / not
10	كما *kamā*	as / also

Table 25: Most frequent sentence-start tokens after missed boundaries.

The repeated markers are informative but not deterministic. Table 27 measures, in train, how often several common sentence-start markers actually follow a gold boundary. Values range from 46.6%

Rank	Spurious-boundary context	English gloss
1	من *min*	from
2	وكان *wa-kāna*	and was
3	فراغ *farāgh*	blank
4	هل *hal*	question particle
5	في *fī*	in
6	ثم *thumma*	then
7	وقد *wa-qad*	and indeed
8	بدلا *badalan*	instead
9	لا *lā*	no / not
10	ولكن *wa-lākin*	but

Table 26: Most frequent token contexts for spurious boundaries.

for ثم *thumma* to 80.3% for فقال *fa-qāla*. None is a reliable boundary rule by itself.

Token	After boundary / total	Rate
فقال *fa-qāla*	57 / 71	**80.3%**
أما *ammā*	46 / 63	73.0%
وقد *wa-qad*	69 / 105	65.7%
هل *hal*	36 / 63	57.1%
ولكن *wa-lākin*	61 / 108	56.5%
ثم *thumma*	88 / 189	46.6%

Table 27: Train-set recurrent boundaries.

A dev-only lexical adjustment shows that simple lookup-based corrections do not resolve the ambiguity. Train-derived token-boundary offsets lower a strong R-Drop AraBERT model from 84.18 to 84.15 dev F1 and lower the SMART model from 84.63 to 84.62. The lexical patterns therefore identify difficult contexts, but simple token rules do not improve the segmentation model.

G.3 A Dev-specific Distribution Shift

The token فراغ *farāgh* ('blank') is the third most frequent false-positive context, accounting for 31 final-system false positives. Its distribution differs sharply across splits. It occurs only twice in train but 142 times in dev, concentrated in five documents. On dev, the true post-token boundary rate is 18.3%, while the final ensemble predicts a boundary after it 81.7% of the time. We therefore treat this as a split-specific distribution shift rather than as evidence for a general Arabic segmentation rule.

Part II: Additional FAAMG Submissions

H Second FAAMG Closed-Track Submission

FAAMG also submitted a second NoPnx-NP closed-track system. It obtains **86.4** document-macro F1 (86.1 precision, 87.8 recall) on the Blind Test set. The system is independent of the heterogeneous ensemble described in the main paper and is built around AraBERT-centered boundary models, family-level score fusion, and document-level calibration. This appendix describes the submitted configuration and the principal experiments that support its design.

H.1 System Architecture

Core AraBERT boundary model. The core model uses AraBERTv02-base (Antoun et al., 2020). AraSeg labels are defined at whitespace-delimited words, whereas AraBERT produces WordPiece representations. For each original word, a learned nonlinear attention module aggregates all of its contextualized subtokens,

$$768 \rightarrow 64 \rightarrow \mathrm{GELU} \rightarrow 1,$$

producing one word representation. A one-dimensional convolution with kernel size 3 and GELU activation adds local word context before a two-class boundary classifier.

Documents are processed with subword-exact overlapping windows of at most 510 content subtokens and stride 128. Complete words are kept together. When a word occurs in multiple windows, its logits are combined with center-weighted pooling, giving greater weight to views in which the word lies farther from a window edge. The final word of every document is marked as a sentence boundary. This window size was fixed before the controlled context-length study reported in Appendix B, which finds that shorter 256-subtoken windows outperform longer alternatives for the AraBERT R-Drop configuration; we did not retune this secondary submission afterward, since

our comparative analysis in the main text centers on the highest-performing submission.

Training uses a clean view and a whole-word-masked view of each batch. Both views are supervised with focal loss (Lin et al., 2017),

$$\mathcal{L} = \mathcal{L}_{\text{focal}}(z_{\text{clean}}, y) + 0.5\,\mathcal{L}_{\text{focal}}(z_{\text{mask}}, y),$$

with $\gamma = 2$. The masking probability increases linearly from 5% to 15% during training. Five models with different random seeds are combined by equal averaging in logit space. Table 28 gives the complete configuration of this five-seed core system.

Setting	Value
Encoder	AraBERTv02-base
Subtoken aggregation	nonlinear attention $768 \to 64 \to 1$
Word-context module	Conv1D, kernel 3 + GELU
Output head	two-class linear classifier
Window / stride	510 / 128 subtokens
Window aggregation	center-weighted logits
Loss	focal, $\gamma = 2$
Masked-view weight	0.5
Whole-word masking	5% $\to$ 15% curriculum
Optimizer steps	4,984
Batch size / accumulation	2 / 2
Learning rate	3×10^{-5}
Schedule	cosine, 10% warmup
Random seeds	13, 42, 43, 44, 45
Seed fusion	equal logit mean
Decision threshold	0.445

Table 28: Configuration of the AraBERT core model in the second FAAMG submission.

Multi-family fusion. The submitted system combines complementary AraBERT model families at score level. Three family scores account for 95% of the fused score after robust-IQR normalization, with equal weights of $19/60$ each. The three families are (i) AraBERTv02-base models trained with differential encoder/head learning rates and plateau scheduling, (ii) AraBERTv02-base models that classify each word from the concatenated context (h_{i-1}, h_i, h_{i+1}) using a $2304 \to 768 \to 2$ GELU MLP, and (iii) AraBERTv02-large boundary models. Family-level aggregation is performed before cross-family fusion so that the number of models within a family does not determine its overall weight.

The remaining $1/20$ of the fused score is provided by a continued AraBERT expert. This expert combines the seed-13 core score with a train-only continued variant in a 0.75/0.25 mixture. The continued model is fine-tuned on the 174 train documents for 156 optimizer steps at learning rate 10^{-5}. Its score is aligned to the scale of the base score with the affine transformation $1.0099195s - 0.0611100$ before interpolation.

Component	Weight
Differential-LR AraBERTv02-base family	$19/60$
Three-word-context AraBERT family	$19/60$
AraBERTv02-large family	$19/60$
Continued AraBERT expert	$1/20$

Table 29: Score-level composition of the second FAAMG submission. Weights operate in robust-IQR-normalized score space.

A document-length correction is applied to the fused score,

$$-0.11875 \log(L/461),$$

where L is the document length in words and 461 is the dev median. A single threshold is calibrated on dev after fusion and length correction, and the final token of every document is forced to a boundary. Table 29 summarizes the submitted score composition.

The five-seed core system reaches 84.74 dev F1 and 85.56 open test F1. The complete system reaches 85.82 dev F1 and 86.54 open test F1, and the corresponding second FAAMG submission obtains 86.4 F1 on the Blind Test set.

H.2 CAMeLBERT Reference Baseline Study

We additionally evaluate CAMeLBERT-MSA (Inoue et al., 2021) under the word-level boundary formulation used for AraSeg. Only the final WordPiece of each original word receives supervision, while all other subtokens and special tokens are ignored by the loss. Inference uses overlapping 512-position windows with tokenizer stride 64 and averages raw logits for words appearing in multiple windows.

The participant train split contains 174 documents and 98,457 words. With effective batch size 32, five epochs correspond to only 60 optimizer updates. We therefore compare the five-epoch configuration with fixed-update training and several inference/ensemble controls. Table 30 reports the full study. Increasing the training budget from 60 to 380 updates raises dev/open test F1 from 74.28/74.35 to 79.70/81.06. The strongest configuration trains five full-data models with different seeds and averages their logits, reaching 80.50 dev and 81.92 open test F1.

Configuration	Eff. batch	Updates	Dev F1	Open F1
Five epochs	32	60	74.28	74.35
Fixed 380-update training	32	380	79.70	81.06
Fixed 380 updates, smaller batch	8	380	79.47	80.69
+ forced final-document boundary	32	380	79.74	81.06
+ Dev-selected threshold	32	380	79.94	81.63
Early-stopping configuration	32	250	79.04	80.07
Center-weighted overlap aggregation	32	380	79.98	81.60
Dense center-position inference	32	380	79.94	–
Five-fold fixed-step logit ensemble	32×5	380×5	79.98	81.64
Five-seed full-data logit ensemble	32×5	380×5	**80.50**	**81.92**

Table 30: CAMeLBERT-MSA boundary-classification study on NoPnx-NP.

Study	Variant	Dev F1	Open F1
Masking granularity	subtoken masking	84.02	84.45
	whole-word masking	**84.28**	83.75
Masked-view objective	KL consistency	84.02	84.45
	dual supervised focal	**84.35**	83.64
Subtoken attention	linear scorer	84.35	83.64
	nonlinear GELU scorer	**84.45**	**84.68**
Classification loss	cross-entropy	84.18	84.00
	focal loss ($\gamma = 2$)	**84.45**	**84.68**
Conv connection	residual Conv1D	83.84	84.02
	plain Conv1D	**84.45**	**84.68**
Conv type	depthwise-separable	83.96	84.03
	standard Conv1D	**84.41**	**84.95**
Training budget	8,912 updates	84.06	84.49
	4,984 updates	**84.41**	**84.95**
Optimization	differential LR + plateau	84.07	84.61
	cosine, 4,984 updates	**84.41**	**84.95**

Table 31: Dev-supported design comparisons for AraBERT.

The full-data five-seed ensemble also exceeds the five-fold ensemble under the same 380-update member budget (80.50/81.92 vs. 79.98/81.64 dev/open test F1), indicating that seed diversity is more useful here than withholding one fold from every member.

H.3 Controlled Design Studies

The AraBERT core architecture is supported by controlled comparisons in which the final choice improves dev F1. Table 31 reports the principal results. Whole-word masking improves over subtoken masking, and direct supervision of the masked view improves over a KL-consistency objective. Nonlinear subtoken attention improves over a linear scorer, and focal loss improves over cross-entropy. For the local-context module, a plain standard Conv1D outperforms residual and depthwise-separable alternatives. The 4,984-update cosine schedule also outperforms both a longer training budget and the tested differential-learning-rate plateau configuration for this model family.

A complementary local-context model concatenates the preceding, current, and following word representations and applies a two-layer GELU classifier ($2304 \to 768 \to 2$). A single model reaches 84.18 dev and 84.44 open test F1. Models of this form constitute one of the three primary families in the submitted multi-family ensemble.

H.4 Seed and Ensemble Effects

The core AraBERT configuration is trained with five random seeds while holding the architecture, loss, masking, optimization budget, and inference procedure fixed. Each model uses the checkpoint with the highest dev document-macro F1. Equal-logit averaging improves over every individual member, reaching 84.74 dev and 85.56 open test F1 (Table 32).

Family-level fusion gives substantially larger gains than the single-family ensemble. Table 33 traces this gain through each added component, from the initial two-family fusion to the final continued-expert blend. Document-length calibration in particular is selected by grouped five-fold dev evaluation rather than the full dev split directly, since its cross-validated score rises from 85.67 to 85.71 before being confirmed on the full dev and open test splits.

H.5 Additional Controls and Negative Results

Several additional methods fail to improve the dev criterion and are excluded from the submitted system (Table 34). A train-fitted expected-boundary-count prior reduces dev F1 for every nonzero mixture weight tested, and additional digit-normalization, Latin-lowercasing, and number-length preprocessing families also fail to improve the dev-selected family fusion.

These results favor neural score fusion over sparse lexical or hand-designed structural corrections. They also motivate a simple final decoder built from score normalization, document-length calibration, a global threshold, and the deterministic final-document boundary.

H.6 Dataset Analysis

Corpus measurements provide additional context for the modeling choices. Table 35 summarizes the principal observations.

Sentence and document lengths. Across the released train, dev, and open test splits, NoPnx-NP contains 35,515 sentence boundaries in total. Beyond the length statistics already given in Table 35, 81.82% of sentences contain at most 16 words and

Model	Dev F1	Open F1	Boundary F1	Precision	Recall
Seed 13	84.41	84.95	85.62	83.47	87.89
Seed 42	84.32	84.90	85.92	85.23	86.63
Seed 43	84.42	84.44	85.56	85.27	85.85
Seed 44	84.05	84.32	85.51	84.91	86.10
Seed 45	84.09	84.69	85.31	82.87	87.90
Equal-logit ensemble	**84.74**	**85.56**	**86.23**	**84.48**	**88.06**

Table 32: Five-seed AraBERT core system. Precision, recall, and boundary F1 are reported on open test.

System	Dev F1	Open F1
Five-seed AraBERT core	84.74	85.56
AraBERT-large / AraBERT-base family fusion	85.16	85.70
Three-family AraBERT fusion	85.54	86.10
Robust-normalized three-family fusion	**85.81**	86.02
+ document-length calibration	85.79	86.35
+ Train-only continued AraBERT expert	**85.82**	**86.54**

Table 33: Representative ensemble configurations for the second FAAMG system (dev-selected; open test reported for reference).

Configuration	Dev F1	Open F1
Ensemble baseline	85.79	86.35
+ 5% sparse-lexical correction	85.78	86.25
Sparse lexical classifier (standalone)	49.57	–
Semi-Markov sentence-length prior (best)	85.78	–
Zero-shot SaT-1L (standalone)	45.11	–

Table 34: Negative-result configurations relative to the ensemble baseline for the second FAAMG submission. None improves over the baseline.

97.33% contain at most 32. This combination of short sentences and long documents favors word-level boundary modeling with overlapping long-document inference rather than hard constraints on sentence length.

Subtoken coverage. Table 35 shows that the large majority of gold sentences fit well within a short subtoken span. The 510-subtoken inference window therefore provides substantially more context than is needed to contain a typical sentence while also supporting long documents without truncation.

Orthographic variation. Appendix B identifies the same class of mechanically repairable spelling forms and explains why we do not normalize them; we report here only the statistics specific to this analysis. Table 35 shows the affected mass is well under 1% of tokens, with both source-specific and standard forms represented by AraBERT without `[UNK]`. Across 133 candidate repair pairs, the source-specific form uses more WordPieces in 115 cases and never fewer, and only four pairs co-occur in the same document, indicating strong source concentration.

I Third FAAMG Closed-Track Submission

FAAMG also submitted a third NoPnx-NP closed-track system. It records **85.5** document-macro F1, with 83.9 precision and 88.3 recall on the leaderboard. The system uses a single AraBERTv02-base encoder with word-level boundary classification, overlapping-window inference, class-weighted training, SMART regularization, and dev-tuned decoding. This appendix gives the complete submitted configuration and its dev behavior.

I.1 System Architecture

The encoder is AraBERTv02-base (Antoun et al., 2020). Each input document is represented as whitespace-delimited AraSeg words and tokenized into WordPieces. The boundary classifier reads the contextual representation of the first subtoken of each original word and applies a linear $768 \to 2$ classification head. For word i with representation h_i, the model predicts

$$p(y_i \mid h_i) = \mathrm{softmax}(W h_i + b),$$

where $y_i = 1$ denotes a sentence boundary after the word.

Documents are processed with overlapping windows of at most 256 subtokens and stride 128. Words that occur in more than one window receive multiple boundary estimates. Their boundary probabilities are averaged before decoding. This preserves coverage of long documents without truncating the document to a single encoder window.

I.2 Training Objective and Configuration

Because sentence boundaries form the minority class, training uses inverse-frequency class

Property	Observation
Boundary balance	10.41% positive labels in Train; 10.15% over Train, Dev, and Open Test.
Sentence length	Median 7 words; 11.34% of sentences contain one word and 9.80% contain two.
Document length	Train median 426.5 words and Dev median 461; maxima 15,472 and 18,606 words.
AraBERT subtoken spans	94.89% of gold sentences fit within 32 subtokens and 99.9831% within 128.
Potential spelling repairs	475 of 349,740 token occurrences (0.136%) fall in a narrow mechanically repairable set.

Table 35: NoPnx-NP corpus measurements for the second FAAMG system.

weights,

$$w_c = \frac{N}{2N_c},$$

inside the two-class cross-entropy objective. The model is additionally regularized with SMART (Jiang et al., 2020). For an input embedding sequence x, SMART searches for a bounded perturbation δ and minimizes the symmetric KL divergence between the clean and perturbed predictions,

$$\mathcal{L} = \mathcal{L}_{\mathrm{CE}} + \lambda_{\mathrm{SMART}}\, D_{\mathrm{KL}}^{\mathrm{sym}}\big(p(\cdot \mid x), p(\cdot \mid x+\delta)\big).$$

Table 36 reports the complete training and inference configuration.

Setting	Value
Encoder	AraBERTv02-base
Word representation	first WordPiece
Output head	linear $768 \to 2$
Window / stride	256 / 128 subtokens
Overlap aggregation	mean boundary probability
Training epochs	12
Batch size	8
Random seed	13
Optimizer	AdamW
Learning rate	2×10^{-5}
Weight decay / warmup	0.01 / 0.10
Schedule	linear decay
Gradient clipping	1.0
Precision	bfloat16
Base loss	class-weighted cross-entropy
SMART steps / LR	2 / 0.03
SMART ϵ / init. magnitude	0.10 / 0.10
SMART weight λ	1.0
Selected τ / γ	0.37 / 0

Table 36: Configuration of the third submission.

I.3 Dev Selection and Decoding

A single checkpoint is selected by dev document-macro F1. Decoding parameters are tuned on the same split. The decision threshold is searched from 0.05 to 0.95 in increments of 0.01, while the minimum allowed gap between consecutive predicted boundaries is selected from $\{0, 2, 3, 4\}$. The selected checkpoint is epoch 11 with $\tau = 0.37$ and $\gamma = 0$. The final decoder therefore applies no minimum-gap restriction.

Epoch	Loss	Dev P	Dev R	Dev F1	τ	γ
1	0.4831	72.68	84.14	76.10	0.71	0
2	0.1514	77.41	88.76	81.69	0.67	0
3	0.0950	80.43	87.65	83.01	0.76	0
4	0.0697	82.58	86.33	83.60	0.68	0
5	0.0520	82.84	86.23	83.83	0.84	0
6	0.0404	82.75	86.92	84.18	0.79	0
7	0.0327	82.78	87.54	84.48	0.65	0
8	0.0285	82.56	87.20	84.19	0.64	0
9	0.0239	82.15	87.90	84.29	0.49	0
10	0.0211	82.74	87.65	84.49	0.45	0
11	0.0187	82.56	87.84	**84.50**	0.37	0
12	0.0176	82.72	87.48	84.42	0.43	0

Table 37: Dev trajectory of the third FAAMG submission. Precision, recall, and F1 are document-macro percentages. τ and γ are the dev-selected decoding parameters for each checkpoint.

Evaluation	Precision	Recall	F1
Dev	82.56	87.84	84.50
Leaderboard	83.90	88.30	**85.50**

Table 38: Document-macro results for the third FAAMG submission.

Table 37 gives the complete dev trajectory. Performance improves rapidly during the first epochs and then stabilizes around 84.2–84.5 F1. The selected epoch reaches 84.50 dev document-macro F1, with 82.56 precision and 87.84 recall, and 85.50 open test F1.

I.4 Results

Table 38 summarizes the selected dev checkpoint and the leaderboard result. The system retains a recall-oriented operating point on both evaluations. The dev search selects no minimum-gap constraint, indicating that simple threshold calibration is preferred over additional structural decoding for this model.